CI-0131 Diseño de Experimentos
Ciclo I-2026
Profesor: Ignacio Díaz Oreiro

Laboratorio #2: Análisis Exploratorio de Datos y gráficos depurados

Introducción

El análisis exploratorio de datos (EDA, por sus siglas en inglés) es un proceso que permite la identificación de las tendencias dentro de los conjuntos de los datos. Durante el desarrollo de un experimento es importante que se logre identificar la presencia de algunas de estas tendencias o medidas como la existencia de valores faltantes, valores extremos, medidas de valores centrales o de dispersión. Esta información valiosa nos da acceso a relevante conocimiento acerca de nuestro conjunto de datos.

Por ejemplo, la presencia de valores faltantes nos puede indicar o permitirnos pensar que durante la recolección de datos algo no fue tan riguroso como se planteó. También que exista algún sesgo en el conjunto de datos, lo que potencialmente podamos evidenciar mediante un análisis visual.

La presencia de valores extremos podría ser un indicador de posibles errores de muestreo, de problemas en la consecución de datos o inclusive de un problema causado por las personas que llevaron a cabo el muestreo. La implementación de una mala estrategia de recolección de datos en un muestreo por parte de un analista puede llevar a un sesgo.

Mediante un análisis exploratorio de datos podemos identificar anomalías y solicitar al ente encargado de ejecutar el estudio que a) rellene los datos faltantes (solicite a los participantes estos datos) o b) brinde la información faltante si fue problema de transcripción. Como vemos el análisis exploratorio nos permite tomar acciones correctivas con respecto a la validez del diseño del experimento, la fase de recolección de datos.

En este laboratorio vamos a trabajar con el conjunto de datos de Kaggle de Pokemon. Puede acceder al siguiente enlace para ver documentación del significado de las columnas:

https://www.kaggle.com/datasets/rounakbanik/pokemon

Trabajo a realizar

Debe crear un reporte con evidencia de que ejecutó todo el Laboratorio. El reporte es un documento con un formato uniforme y coherente, en el que se incluye texto explicativo del trabajo que se va realizando, los segmentos de código R y los resultados R que se van aportando como parte del laboratorio. El reporte debe poder leerlo y entenderlo una persona que no cuenta con el enunciado del laboratorio.

Para los gráficos en los que se le proveyó de código, debe incluir el grafico en el reporte, debidamente rotulado con la información que presenta.

Para los gráficos donde usted debe crear las instrucciones en R (que inician con "**Construya**…") debe presentar tanto el grafico resultante como el código R que utilizó para crearlo.

Cada gráfico y resultado debe estar debidamente identificado. Esto debe hacerse uno por uno, no es válido copiar todo el código R y luego todos los resultados y gráficos. Debe indicar también qué acción está realizando e identificar claramente los resultados que presenta R.

En algunas secciones se le realizarán preguntas que debe contestar e incluir en el reporte. Estas aparecen Precedidas del texto "**Pregunta**" en negrita.

El laboratorio se divide en dos partes: en la primera se realizará un análisis exploratorio de datos, con resultados y gráficos sencillos (pero que deben contar con toda la información pertinente y ser legibles en el tamaño de letra y otros elementos), mientras que en la segunda parte se realizarán algunos gráficos con formato más depurado. Para la primera parte se utilizarán gráficos y facilidades que provee R, mientras que para la segunda parte es necesario cargar algunas bibliotecas.

Instrucciones de la entrega:

El trabajo se entrega en grupos de 2 ó 3 personas. Basta con que uno de los integrantes suba la solución a Mediación Virtual.

Las entregas se realizarán en formato PDF. La letra debe ser Times New Roman, Arial o Cambria. Los tamaños permitidos son 10-12.

Debe incluir una portada de página completa en la que incluya el nombre del curso, del laboratorio, así como los nombres completos de los estudiantes. Esta página será utilizada por el profesor para apuntar la nota y también para incluir comentarios en caso de tener que incluir generales del informe.

El reporte se debe entregar en Mediación Virtual, en un archivo .pdf que pueda ser leído en programas comerciales de uso habitual. Debe verificar que el .pdf que subió a Mediación Virtual contiene los ejercicios resueltos y que el archivo puede abrirse correctamente. En caso de problemas con el archivo .pdf (no abre correctamente, está corrupto, etc.) se considerará que no entregó la tarea.

Las imágenes deben ser visibles y contar con todos los elementos necesarios para su entendimiento, por ejemplo, que las escalas estén completas, que sean legibles, etc. Se recomienda ampliar la imagen en RStudio antes de copiarla en el informe.

En los casos que deba incluir código R en el reporte, cópielo como un pantallazo (screenshot) o copiando el texto, pero de forma que no se pierda el formato original.

Las entregas tardías se penalizarán con un 10% de la nota luego de vencida la fecha y hora de entrega, más un 10% adicional por cada hora de retraso.

A. Primera parte

Todos los gráficos que se muestran o solicitan generar deben incluirse en el Reporte de Laboratorio. Para los gráficos para los que no se provee el código (identificados con la palabra ***Construya***), usted debe incluir en el texto del reporte el código R que usted creó (no como un archivo separado).

Además, todos los resultados que se generen a partir de código en R (se despliegan en la consola) deben incluirse en el reporte.

A.1. Información general

Lea el CSV que se proveyó con el enunciado (pokemon.csv) con la información de los Pokémon:

Ejecute primero la instrucción *df <- (read.csv(file.choose…* . Esta instrucción abrirá un diálogo de Windows que le permitirá seleccionar el archivo a cargar y a asociar a la variable df.

Luego ejecute la instrucción attach(df)

```
df= (read.csv(file.choose(), header=T, encoding = "UTF-8"))

# Luego de cargado el archive ejecute la función attach, que le permite acceder a columnas del dataframe
# con solo el nombre
attach(df)
```

Al usar file.choose() usted puede elegir el archivo sin necesidad de definir la ruta. Además, incluir header y encoding le ayudan a read.csv a identificar cabeceras y el formato del texto

El comando summary() nos brinda información de las variables, como cuartiles y datos máximos y mínimos.

Ahora bien, incluya los datos para las variables que más nos interesan en este laboratorio, a saber: **attack, defense, hp, weight_kg y height_m.**

Para cada una de ellas **construya**, ejecute y copie la salida del comando summary() en el reporte.

El texto a incluir en el informe debe ser explicativo para este ejercicio y los que siguen. Por ejemplo:

> Se presenta seguidamente la información de la columna attack.
>
> El código R que se ejecuta es el siguiente:
>
> ```
> #
> summary(df$attack)
> ```
>
> Se obtuvo el siguiente resultado:
>
> ```
> > summary(df$attack)
>  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
>  5.00   55.00   75.00   77.86  100.00  185.00
> ```

Ahora siga el mismo formato para las otras variables que se indicaron arriba.

Pregunta: ¿Encontró algunos valores faltantes? ¿En qué variables?

Para variables categóricas se puede obtener información utilizando el comando table(). A estas tablas se les llama tablas de contingencia.

Encuentre las cantidades de Pokémon por tipo para type1 y type2. Para eso **construya** las tablas de contingencia para estas dos variables.

```
table(df$type1)

table(df$type2)
```

También **construya** tablas de contingencia para las variables is_legendary y generation.

Ahora bien, también es posible utilizar la instrucción summary para las variables categóricas, pero hay que indicarle a R que se trata de variables de este tipo. Para ello se puede usar la instrucción *as.factor()*.

```
# Por ejemplo, para la variable type1

df$type1 <- as.factor(df$type1)

# Note que la instrucción anterior modifica los valores en el data frame !

summary(df$type1)
```

```
> summary(df$type1)
     bug     dark   dragon electric    fairy fighting     fire   flying    ghost 
      72       29       27       39       18       28       52        3       27 
   grass   ground      ice   normal   poison  psychic     rock    steel    water 
      78       32       23      105       32       53       45       24      114 
```

Construya ahora instrucciones equivalentes as.factor() y summary() para obtener resultados equivalentes para las tablas de contingencia anteriores (para las variables type2, generation, is_legendary).

Las tablas de contingencia también permiten clasificar la información para dos variables. Por ejemplo, separando los Pokémon legendarios o no según la generación.

Por ejemplo, una tabla de contingencia que represente cuántos Pokémon de cada generación (generation) son legendarios o no (is_legendary).

Se puede hacer de dos maneras:

```
> table(generation, is_legendary)
          is_legendary
generation   0   1
         1 146   5
         2  94   6
         3 125  10
         4  94  13
         5 143  13
         6  66   6
         7  63  17
> table(is_legendary, generation)
            generation
is_legendary   1   2   3   4   5   6   7
           0 146  94 125  94 143  66  63
           1   5   6  10  13  13   6  17
```

Pregunta: ¿Cuáles son los 3 grupos más numerosos de esa última tabla?

Construya ahora cuatro nuevas tablas que combinen las variables categóricas type1 con generation, type2 con generation, type1 con is_legendary y type2 con is_legendary.

Pregunta: De los Pokémon tipo 1, ¿cuál es el grupo más numeroso de Pokémon legendarios en este set de datos?

En R es posible aplicar una función a un conjunto de datos clasificados. Por ejemplo, el siguiente código genera la media de la variable hp, clasificando los datos por generación.

Para construir tablas que clasifiquen variables discretas y continuas se puede usar *tapply*().

Por ejemplo, el siguiente código genera la media de la variable hp, clasificado por generación.

```
tapply(df$hp, df$generation, mean)
```

```
> tapply(df$hp, df$generation, mean)
       1        2        3        4        5        6        7
64.34437 70.98000 65.66667 73.10280 70.31410 71.11111 70.57500
```

Otra forma es utilizar la biblioteca dyplr.

En este mismo caso, otra forma de hacerlo sería:

```
df %>%
 group_by(generation) %>%
 summarise(mean_hp = mean(hp, na.rm = TRUE))
```

```
> df %>%
+   group_by(generation) %>%
+   summarise(mean_hp = mean(hp, na.rm = TRUE))
# A tibble: 7 × 2
  generation mean_hp
       <int>   <dbl>
1          1    64.3
2          2    71.0
3          3    65.7
4          4    73.1
5          5    70.3
6          6    71.1
7          7    70.6
```

La salida se presenta como un "tibble" que es una estructura similar a un data frame.
El incluir "na.rm = TRUE" hace que se eliminen del cálculo los valores faltantes NA.

Construya dos tablas (tibble de dyplr) que calculen la media de weight_kg por generación. La primera removiendo los NA y la segunda sin removerlos.

Pregunta: ¿Qué pasa cuando una columna sí tiene NAs y no se remueven al construir la tabla?

A.2. Histogramas

Los histogramas nos brindan información sobre variables continuas.

```
hist(df$hp)

# Se puede hacer un poco más claro agregando título y etiquetas:
hist(df$hp, main="Distribución de variable hp", xlab="HP", col="lightblue", border="black")
```

Construya histogramas similares para las variables attack, defense, weight_kg y height_m.

Pueden ser histogramas **simples**, dado que estamos en la fase de exploración. No son los que eventualmente se incluirán en reportes o artículos.

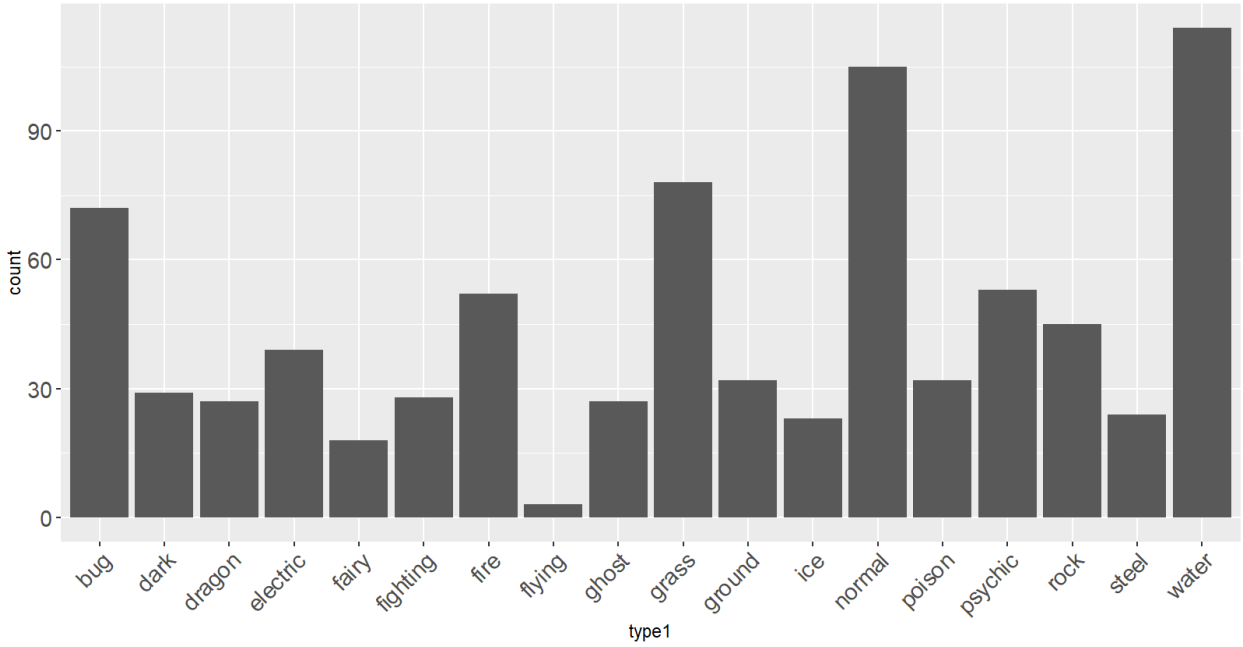
A.3. Gráficos de barras

Los gráficos de barras nos brindan información sobre variables discretas.

Se puede construir un gráfico de barras para los Pokémon de tipo 1 con el siguiente código:

```
#
ggplot(pokemon, aes(x = type1)) +
  geom_bar() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

El resultado es el siguiente:



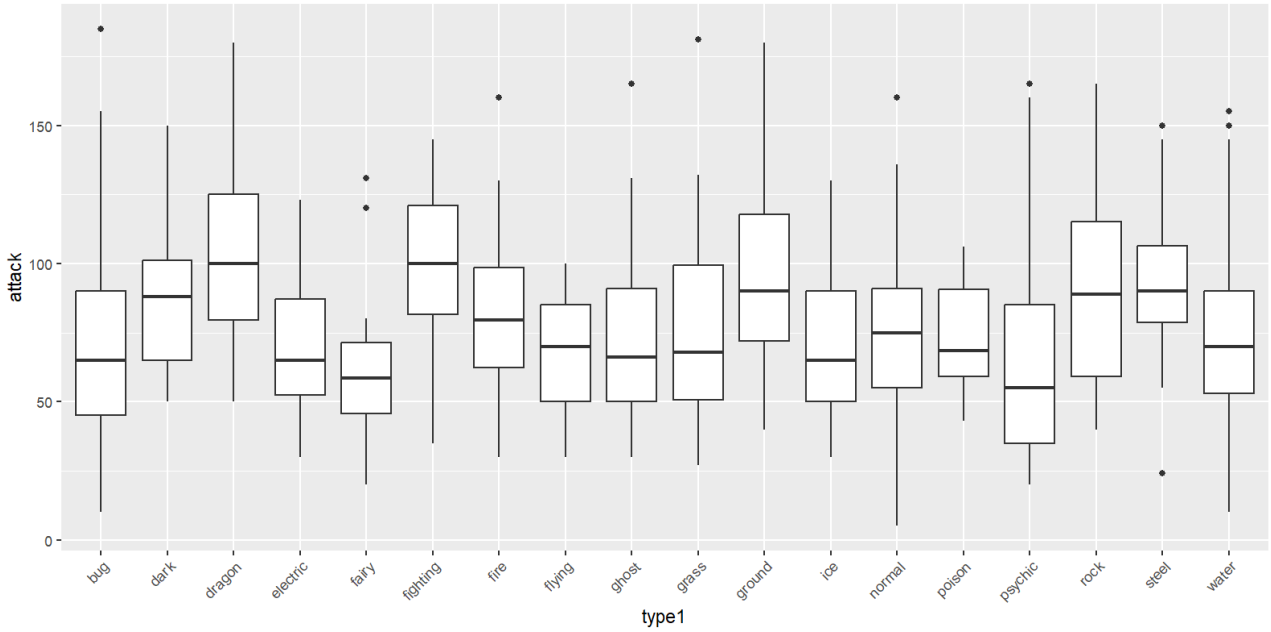
Construya gráficos de barras para las variables type2, is_legendary y generation. Tenga cuidado que los elementos del gráfico sean legibles.

A.4. Boxplots

Podemos construir boxplots para comparar valores de una variable continua con una categórica. Por ejemplo, graficar los valores de attack por tipo de Pokémon para los Pokémon tipo 1.

```
#
ggplot(pokemon, aes(x = type1, y = attack)) +
  geom_boxplot() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

El resultado es el siguiente:



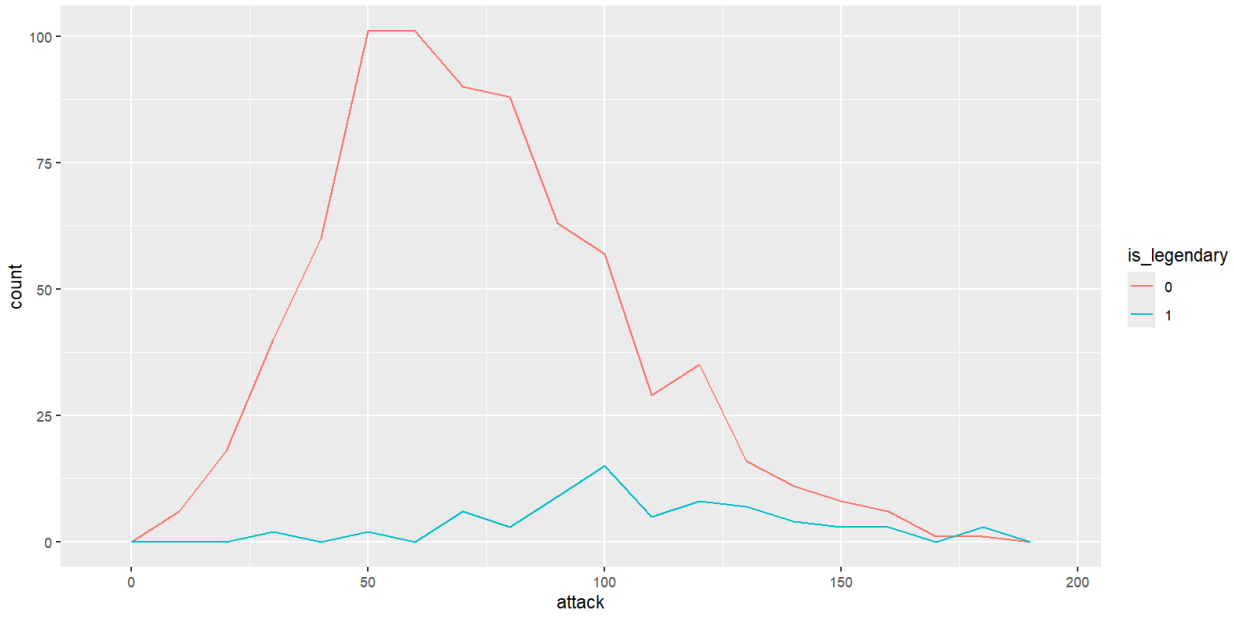
Construya boxplot similares para ataque versus generation, defense versus tipo 1, ataque versus tipo2 y hp versus tipo1.

A.5. Polígonos de frecuencia

Otra forma de graficar una variable continua versus una categórica es mediante los polígonos de frecuencia. El siguiente gráfico, por ejemplo, sirve para comparar el ataque de los Pokémon legendarios y no legendarios:

```
#
ggplot(pokemon, aes(x = attack, color = is_legendary)) +
  geom_freqpoly(binwidth = 10)
```

El resultado es el siguiente:



Construya gráficos similares para defense versus generation, hp versus generation y weight_kg versus is_legendary.

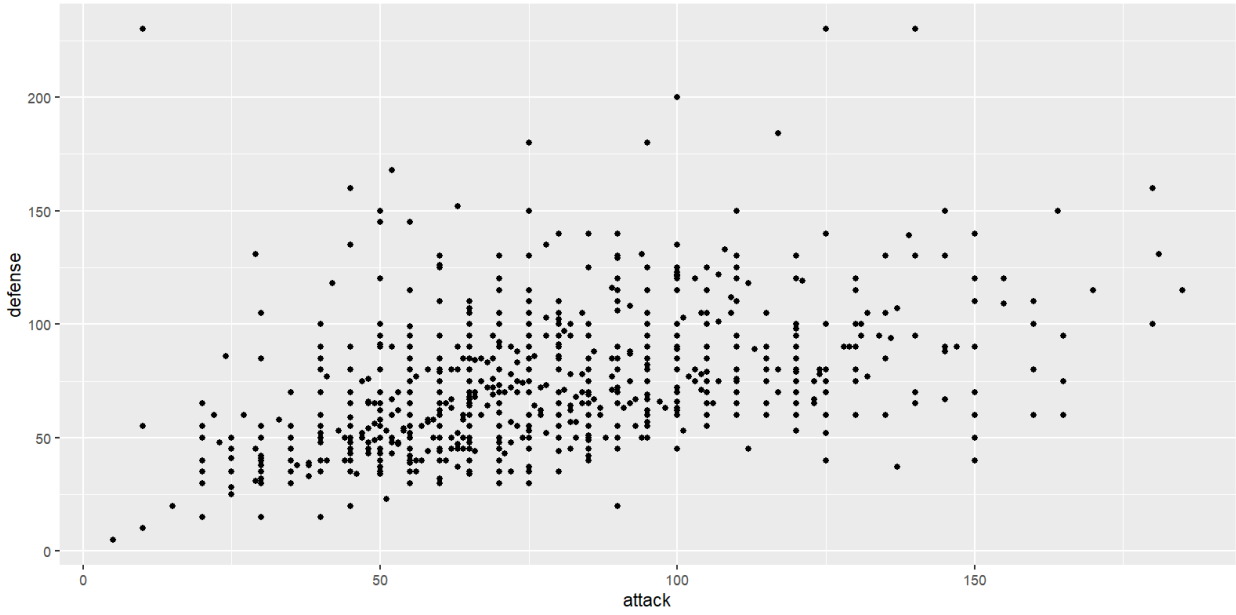
A.6. Gráficos de dispersión

Podemos usar estos gráficos para ver cómo se comportan dos variables, si por ejemplo se sospecha que puede haber algún tipo de correlación.

Por ejemplo, el siguiente código construye un gráfico de dispersión entre attack y defense.

```
ggplot(pokemon, aes(x = attack, y = defense)) +
   geom_point()
```

El resultado es el siguiente:



Construya diagramas de dispersión para similares para las variables attack versus speed, para speed versus defense y speed versus hp.

También es posible generara varios gráficos en uno solo, para tener una vista general de varios, y luego estudiar los más interesantes. **Construya** la matriz de dispersión con el siguiente código R.

```
# Matrices de dispersión dos a dos para hp, attack, defense y speed.

pairs(df[, c("hp", "attack", "speed")], main="Matrices de Dispersión")
```

B. Segunda parte

Todos los gráficos que se muestran o solicitan generar deben incluirse en el Reporte de Laboratorio, con el respectivo texto que indica qué se va a hacer y el resultado obtenido. Para los gráficos para los que no se provee el código (identificados con la palabra ***Construya***), usted debe incluir también en el reporte el código R que usted creó.

Instalar los paquetes:

```
install.packages(c("dplyr",
                   "ggplot2",
                   "gridExtra",
                   "tidyr",
                    "reshape2",
                    "RColorBrewer",
                   "ggrepel"))
```

Cargar los paquetes:

```
library(dplyr)
library(ggplot2)
library(gridExtra)
library(tidyr)
library(reshape2)
library(RColorBrewer)
library(ggrepel)
```

Definimos el conjunto como una tibble que es un data frame simplificado.

Las tibbles son versiones de dataframes con algunas facilidades de impresión y uso.

```
df = tibble::as_tibble(df)
colnames(df)[25] <- "classification"
df$capture_rate <- as.numeric(df$capture_rate)
```

Como se puede ver el data frame original es bastante extenso entonces para esta segunda parte se van a seleccionar algunas columnas para realizar las exploración y visualizaciones.

```
df = select(df, name, classification, hp, weight_kg,
        height_m, speed, attack, defense,
        sp_attack, sp_defense, type1, type2,
        abilities, generation, is_legendary,
        capture_rate)

head(df)
```

No es necesario incluir en el reporte el resultado del head(df). Se muestra para que se noten las columnas seleccionadas.

B.1. Gráficos de densidad de varios atributos de Pokémon.

Un diagrama de densidad visualiza la distribución de datos en un intervalo continuo. Este gráfico es una variación de un histograma que utiliza el suavizado del núcleo para trazar valores, lo que permite distribuciones más uniformes al suavizar el ruido. Los picos de un gráfico de densidad ayudan a mostrar dónde se concentran los valores durante el intervalo.

```
density_hp <- ggplot(data=df, aes(hp)) + geom_density(col="white",fill="brown", alpha=0.8) +
ggtitle("Densidad de Hit Points o Vida")
density_speed <- ggplot(data=df, aes(speed)) + geom_density(col="white", fill="blue", alpha=0.8) +
ggtitle("Densidad de velocidad")
density_attack <- ggplot(data=df, aes(attack)) + geom_density(col="white", fill="orange", alpha=0.7) +
ggtitle("Densidad caracteristicas ofensivas")
density_defense <- ggplot(data=df, aes(defense)) + geom_density(col="white", fill="yellow", alpha=0.7) +
ggtitle("Densidad caracteristicas defensivas")
grid.arrange(density_speed, density_hp, density_attack, density_defense, ncol=2)
```

El gráfico anterior lógicamente debe incluirlo en el reporte.

Construya un nuevo diagrama:

Modifique el grid anterior para incluir dos diagramas de densidad más:

a. El diagrama de densidad basado en la altura (m) que utiliza la variable height_m
b. El diagrama de densidad de peso (kg) que utiliza la variable weight_kg

El nuevo grid contará ahora con 6 diagramas. Asegúrese que el grid se presenta ahora en 3 columnas, y que los textos y títulos se despliegan apropiadamente.

B.2. Diagramas de Caja o Boxplots

Es posible crear boxplots con más detalle y que permiten comparaciones de múltiples variables a la vez. Por ejemplo, el siguiente código permite comparar varias características (hp, defense, attack, sp_attack, sp_defense, speed) para Pokémon legendarios o no.

```
box_plot_attr <- select(df, type1, is_legendary, hp, defense, attack, sp_attack, sp_defense, speed)

box_plot_attr_leg <- filter(box_plot_attr, is_legendary == 1)

box_plot_attr_nor <- filter(box_plot_attr, is_legendary == 0)

box_plot_attr_leg_long <- gather(box_plot_attr_leg, attribute, value, -c(type1, is_legendary))

box_plot_attr_nor_long <- gather(box_plot_attr_nor, attribute, value, -c(type1, is_legendary))

bp_leg <- ggplot(data = box_plot_attr_leg_long, aes(attribute, value)) + geom_boxplot(fill="green4") +
ggtitle("Pokemon Legendario")

bp_nor <- ggplot(data = box_plot_attr_nor_long, aes(attribute, value)) + geom_boxplot(fill = "yellow2") +
ggtitle("Pokemon No Legendario")

grid.arrange(bp_leg, bp_nor,ncol=2)
```

B.3. Mapas de calor

Un mapa de calor se puede definir como tablas con colores en lugar de números, donde una escala colorimétrica representa un rango de valores continuos (escala de grises en lugar de valores de 0 a 1). Con base en esto podemos entender entonces que por ejemplo el valor más bajo se establece en azul oscuro, el valor más alto en rojo y los valores de rango medio en blanco, con una transición (o gradiente) correspondiente entre estos extremos.

Los mapas de calor son muy usados para visualizar cantidades de datos multidimensionales. Esta característica los hace parte del conjunto de herramientas que permiten explorar e identificar tendencias relevantes en un conjunto de datos. Por ejemplo, a través de los mapas de calor podemos identificar grupos

de filas con valores similares, ya que se muestran como áreas de color similar. Esto permite identificar variables que poseen una relación.

A continuación, veremos cómo los atributos de varios tipos de Pokémon se comparan entre sí. Para esto, calcularemos la mediana de estos atributos y trazaremos un mapa de calor para Pokémon legendarios y no legendarios.

Mapa de calor de tipo primario vs atributo - Pokémon legendarios

En este momento utilizaremos mapas de calor para analizar los Pokemon tipo hielo. Este análisis nos permite ver que los Pokemon legendarios de tipo hielo o Ice tienen un valor muy alto de defensa especial (sp_defense), pero un valor bajo de velocidad o Speed. Asimismo, podemos apreciar que los Pokémon legendarios terrestres tienen un atributo de ataque muy alto, y aquellos Pokémon legendarios de tipo bug o Insecto tienen un nivel muy bajo de sp_defense o defensa especial.

```
hmap_attr <- select(df, type1, is_legendary, hp, defense, attack, sp_attack, sp_defense, speed)
hmap_attr_leg <- filter(hmap_attr, is_legendary == 1)
hmap_attr_leg <- group_by(hmap_attr_leg, type1)
hmap_attr_leg <- summarise(hmap_attr_leg, hp=median(hp), attack=median(attack),
defense=median(defense), sp_attack=median(sp_attack), sp_defense=median(sp_defense),
speed=median(speed))

hmap_attr_leg_m <- melt(hmap_attr_leg)
hm.palette <- colorRampPalette(rev(brewer.pal(5, 'RdYlBu')), space='Lab')

ggplot(data=hmap_attr_leg_m, aes(type1, variable)) + geom_tile(aes(fill=value)) + ggtitle("Pokémon
Legendario: Type1 - Atributo") + scale_fill_gradientn(colours = hm.palette(100)) + theme(axis.text.x =
element_text(angle=90, hjust=0)) + coord_equal()
```

Mapa de calor de tipo primario vs atributo - Pokémon no legendarios

Construya un mapa de calor equivalente, pero para Pokémon no legendarios.

En este escenario podemos ver que:

a. Los Pokémon tipo acero tienen un alto valor de defensa, pero muy baja velocidad.
b. Los Pokémon de tipo lucha (fighting) tienen un alto valor de ataque, pero un valor muy bajo de sp_attack.
c. Los Pokémon terrestres, de hielo y normales tienen niveles de HP similares
d. Los Pokémon de tipo veneno, psíquico y de roca tienen niveles de HP similar.

Fin del laboratorio